

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовая работа
по курсу «Параллельная обработка данных»

Обратная трассировка лучей (Ray Tracing) на GPU.

Выполнил: Орусский В.Р.
Группа: М8О-406Б-21
Преподаватели: А.Ю. Морозов

Москва, 2025

Цель работы: Использование GPU для создание фотореалистической визуализации. Рендеринг полужеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.

Вариант 3: Тетраэдр, Гексаэдр, Икосаэдр.

Программное и аппаратное обеспечение

GPU:

- Название NVIDIA GeForce GTX 1660 TI
- Compute capability: 7.5
- Графическая память: 2147155968
- Разделяемая память: 49152
- Константная память: 65536
- Количество регистров на блок: 32
- Максимальное количество нитей: (1024, 1024, 64)
- Максимальное количество блоков: (2147483647, 65535, 65535)
- Количество мультипроцессоров: 24

Сведения о системе:

- Процессор: Intel(R) Core(TM) i5-10600KF CPU @ 4.5000MHz
- ОЗУ: 16 ГБ
- SSD M2 1 ТБ

Программное обеспечение:

- OS: Windows 10 + WSL (Ubuntu 20.04)
- IDE: Visual Studio Code
- Компилятор: nvcc

Метод решения

Для отрисовки используются классические треугольные полигоны, которые состоят из векторов RGBA пикселей. Для всех 3 фигур и пола (платформы) потребуется 38 полигонов. Рендеринг кадров и сглаживание выполним параллельно для разных пикселей посредством технологий CUDA. Отрисовка кадра будет реализована, как на хосте, так и на девайсе, в зависимости от переданного аргумента при запуске.

Описание программы

- namespace tools: внутри этого пространства имён реализованы функции для взаимодействия с векторами (норма, длина, умножение на число, скалярное произведение)
- Triangle: структура для хранения треугольника (полигона)
- Ray: функция, которая вычисляет цвет пикселя в рамках трассировки луча (первое пересечение, учёт направления камеры, освещения и тд)
- cpuRender / gpuRender: Создание кадра сцены на хосте и девайсе соответственно.
- cpuSmoothing / gpuSmoothing: Сглаживание текстур с помощью усреднения значения пикселей. Соответственно выполняется на хосте и девайсе.

- Класс SceneRenderer: Отвечает за создание сцены (создание фигур, конфигов для различных объектов и тд)
- namespace configs: Пространство имён для всех конфигов (фигуры, камера, освещение, пол)

Взаимодействие с программой

Программа поддерживает следующие ключи:

- **--default** — использование конфига по умолчанию;
- **--cpu** — запуск рендеринга на хосте;
- **--gpu** — запуск рендеринга на GPU. Используется по умолчанию.

Консольный ввод (если не использовать флаги):

1. Количество кадров в конечной анимации;
 2. Путь к выходным изображениям;
 3. Разрешение кадра и угол обзора;
 4. Параметры движения камеры (20 вещественных чисел);
 5. Параметры тетраэдра: координаты центра, цвет, радиус описанной окружности;
 6. Параметры гексаэдра: координаты центра, цвет, радиус описанной окружности;
 7. Параметры икосаэдра: координаты центра, цвет, радиус описанной окружности;
 8. Параметры платформы: координаты четырёх точек, цвет;
 9. параметры источника света: положение, цвет, квадратный корень из количества лучей на один пиксель;
- Координата – 3 вещественных числа;
 - Цвет – 3 вещественных числа;
 - Радиус – 1 вещественное число

Пример входных данных:

360

%d.data

800 800 90

7, 3, 0, 2, 1, 2, 6, 1, 0, 5, 2, 0, 3, 0.5, 0.1, 2, 4, 1, 0, 0

0, -2, 0 1, 1, 100, 255 1

0, 0, 0 255, 255, 0, 255 1

0, 2, 0 255, 255, 255, 255 1

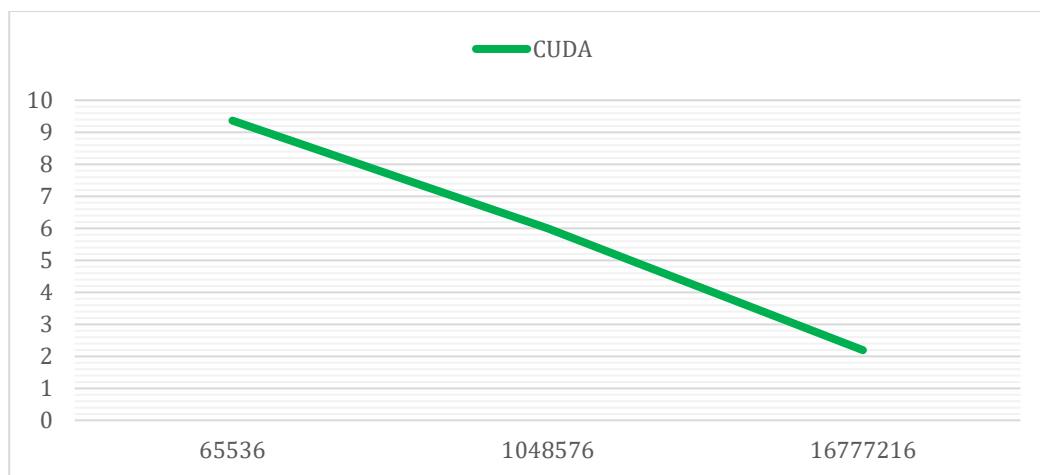
-5, -5, -1 -5, 5, -1 5, 5, -1 5, -5, -1 255, 0, 0, 255

10, 0, 15 75, 50, 25, 255 4

Результаты

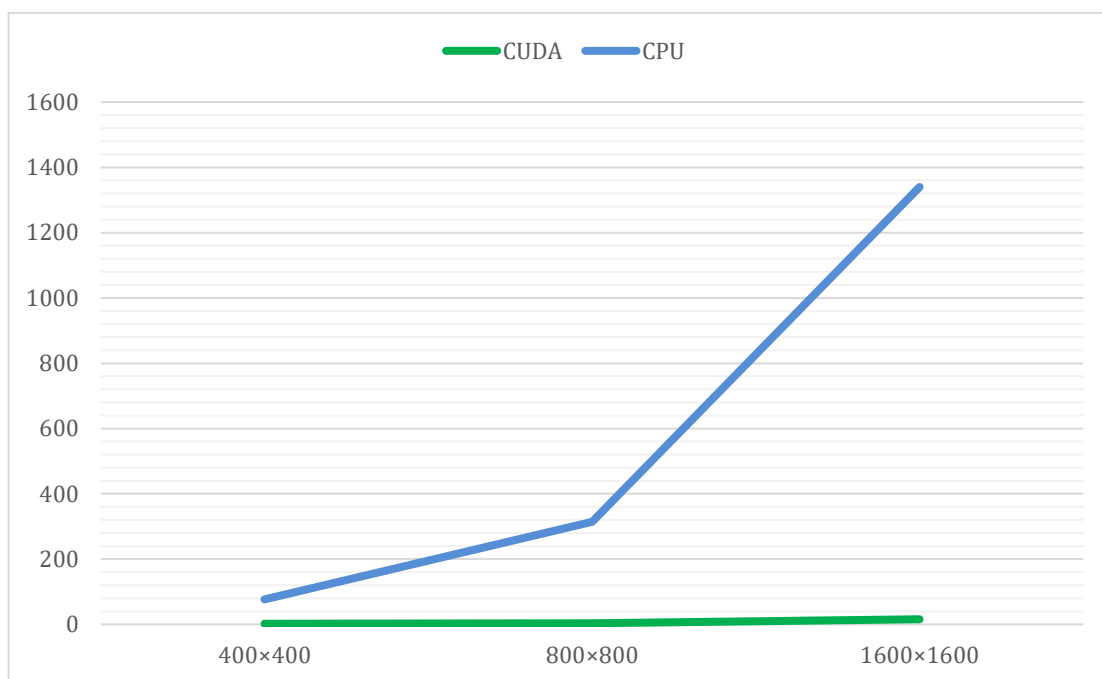
1. Зависимость времени от потоков:

Потоки	Время (в сек)
16×16×16×16	9,3641
32×32×32×32	6,0001
64×64×64×64	2,1922

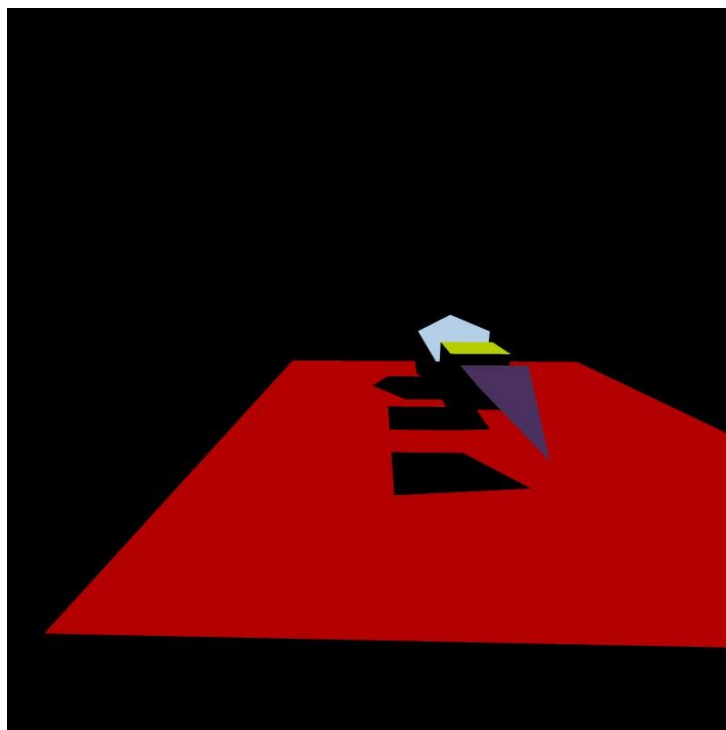
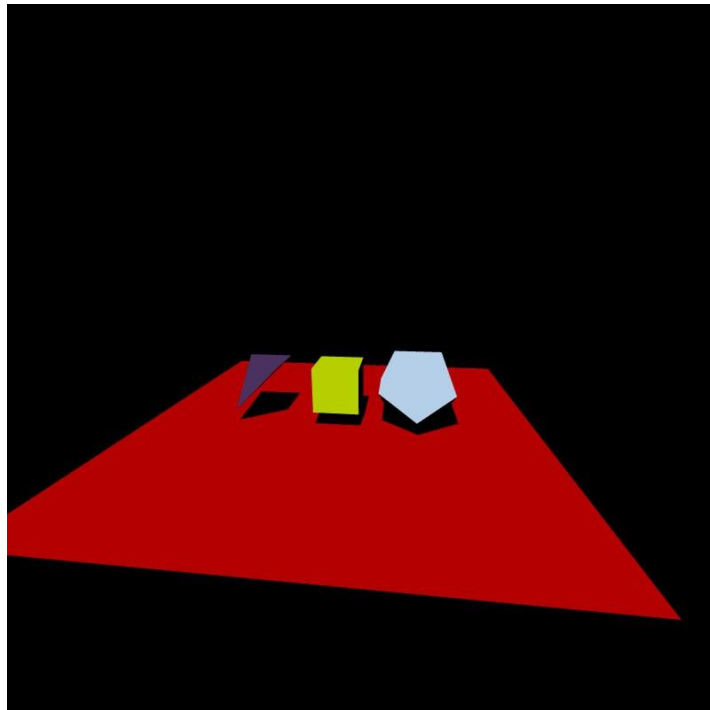


2. Сравнение программы на CUDA с $8 \times 8 \times 8 \times 8$ потоками и программы на CPU с одним потоком (рендеринг 25 кадров):

Размер кадра	Время на CUDA (в сек)	Время на CPU (в сек)
400×400	1,8401	76,5154
800×800	3,7164	314,3254
1600×1600	15,7010	1339,8690



1. Полученное изображение:



Выводы

В ходе выполнения лабораторной работы я реализовал рендеринг изображений с использованием GPU, работал с полигонами, освещением и сглаживанием, а также сравнил скорость рендеринга на CPU и GPU. В очередной раз убедился, что видеокарты значительно превосходят процессоры в задачах, требующих параллельных вычислений. В результате получил изображение трёх объемных фигур на поверхности с реалистичным

освещением и тенями. Было интересно узнать о том, как работает рейтрейсинг, я понял, почему во многих играх так сложно строить реалистичные, особенно динамичные отражения.

Литература:

1. Документация CUDA от Nvidia, URL: <https://developer.nvidia.com/cuda-toolkit>
2. Блог разработчиков NVIDIA, URL: <https://developer.nvidia.com/blog/accelerated-ray-tracing-cuda/>
3. Написание трассировки лучей на CUDA, URL: <https://retroscience.net/ray-tracing-with-cuda.html>
4. Икосаэдр, URL: <https://ru.wikipedia.org/wiki/%D0%98%D0%BA%D0%BE%D1%81%D0%B0%D1%8D%D0%B4%D1%80>